

→ Size of the array is  $n$ .

$$1 < n < 10^5$$

→  $10^8$  iterations  $\Rightarrow$  1 sec.

→ Your code should get executed  $\leq$  1 sec.

→ If your code takes more than 1 sec

↓  
TLE [ Time Limit Exceeded ]

| $n \rightarrow$ | TC       | Total iterations | TLE               |
|-----------------|----------|------------------|-------------------|
| $10^2$          | $O(n^5)$ | $10^{10}$        | ✓                 |
| $10^2$          | $O(n^4)$ | $10^8$           | X (on the border) |
| $10^2$          | $O(n^3)$ | $10^6$           | X                 |
| $10^3$          | $O(n^2)$ | $10^6$           | /                 |

- Read the **Question** and **Constraints** carefully.
- Formulate an **Idea** or **Logic**.
- Verify the **Correctness** of the Logic.  $\rightarrow$  by trying on Test Cases
- Mentally develop a **Pseudocode** or rough **Idea of Loops**.
- Determine the **Time Complexity** based on the Pseudocode.
- Assess if the time complexity is feasible and won't result in **Time Limit Exceeded (TLE)** errors.



Space Complexity: Space taken (worst case) by your algorithm excluding the input & the output space.

func (int n) {

int x; // 4 bytes

int y; // 4 bytes

long z; // 8 bytes.

}

$$4 + 4 + 8$$

$$= 16 \Rightarrow O(1)$$

func (int n) {

int arr[n]; // 4n bytes

int x; // 4 bytes

int y; // 4 bytes

long z; // 8 bytes.

}

$$4n + 16$$

$$SC: O(n)$$

Space Complexity

At max

$\supseteq$

$10^6$  integers

SC:  $O(n^2)$

$$1 \leq n \leq 10^2$$

$\Rightarrow$

$10^2$

Find the Space Complexity [Big(O)] of the below program.

```
func(int N) { // 4 bytes
    int arr[10]; // 40 Bytes
    int x; // 4 bytes
    int y; // 4 bytes
    long z; // 8 bytes
    int arr[N]; // 4 * N bytes
}
```

SC:  $O(N)$

Find the Space Complexity [Big(O)] of the below program.

```
func(int N) { // 4 bytes
    int x = N; // 4 bytes
    int y = x * x; // 4 bytes
    long z = x + y; // 8 bytes
    int arr[N]; // 4 * N bytes
    long l[N][N]; // 8 * N * N bytes
}
```

SC:  $O(N^2)$

Find the Space Complexity [Big(O)] of the below program.

```
function maxArr(int arr[], int N) {
    int ans = arr[0];
    for(i -> 1 to N-1) {
        ans = max(ans, arr[i]);
    }
    return ans;
}
```

SC:  $O(1)$

$$\Rightarrow \cancel{5n^2 + 3n + 100}$$

$$\Rightarrow \underline{\underline{O(n^2)}}$$

```
Function mysteryFunction(N):
```

```
    result = 0
```

```
    // First loop
```

```
    For i from 1 to N:
```

```
        For j from 1 to N:
```

```
            result = result + 1
```

```
    // Second loop
```

```
    For i from 1 to N:
```

```
        result = result + log(i)
```

```
    // Third loop
```

```
    For i from 1 to  $2^N$ :
```

```
        result = result + 1
```

```
    Return result
```

$$\cancel{I n^2 +}$$

$$\cancel{I n +}$$

$$\cancel{I 2^N} = \underline{\underline{O(2^N)}}$$

## What is the Time complexity?

```
Function solve(N, M):
```

```
    For i from 1 to N:
```

```
If N mod i == 0:  
    Print i  
  
For i from 1 to M:  
    If M mod i == 0:  
        Print i
```

$\Rightarrow$   $\text{for } (\text{int } i=0 ; i \leq n ; i++)$   $\ll$   $//n$   
3

$\text{for } (\text{int } j=0 ; j \leq m ; j++)$   $\ll$   $//m$   
3

$TC : O(n+m)$

for (int i = 0 ; i < n ; i++) L.

for (int j = 0 ; j < m ; j++) L

==

}  
}

TC :  $O(mn)$

**What is the Time complexity?**

**Function** func(n):

```
s = 0
For i from 1 to 100:
    s = s + i
Return s
```

100  
 $\Rightarrow \underline{\underline{O(1)}}$

4 options

**What is the Time Complexity of the code?**

```
Function func(n):
    s = 0
    i = 0
    While i < n:
        s = s + i
        i = i * 2
    Return s
```

$\Rightarrow \underline{\underline{O(\log n)}}$

for (int i = 0 ; i < n ; i++) {

for (int j = 0 ; j ≤ i ; j++) {

Print (i+j);

}

| i   | j        | Total iterations |
|-----|----------|------------------|
| 0   | [0, 0]   | 1                |
| 1   | [0, 1]   | 2                |
| 2   | [0, 2]   | 3                |
| 3   | [0, 3]   | 4                |
| ⋮   |          | ⋮                |
| n-1 | [0, n-1] | n                |

$$\frac{n(n+1)}{2}$$

$$\frac{n^2}{2} + \frac{n}{2}$$

$$\Rightarrow \underline{\underline{O(n^2)}}$$

for (int i = 1; i ≤ 100; i = i \* 2)

for (int j = 1; j ≤ n; j++)

print(i \* j)

$$C \times n \approx \underline{\underline{O(n)}}$$

$$\log n < 2^n < n! < n^n$$

Function f1(n):

result = 0

For i from 1 to n:

j = 1

While j < n:

result = result + 1

j = j \* 2

Return result

$$P_1 = \underline{\underline{n \log n}}$$

Function f2(n):

result = 1

For i from 1 to n:

result = result \* 2

Return result

$$P_2 = O(n)$$

Function f3(n):

result = 1

For i from 1 to n:

For j from 1 to n:

result = result \* n

Return result

$$P_3 = O(n^2)$$

Function f4(n):

count = 0

i = 1

While i <= n:

count = count + 1

i = i \* 2

Return count

$$P_4 = O(\log n)$$

$P_4, P_2, P_1, P_3$

---

# For (int i = 1 ; i ≤ n ; i = i \* 2) {

}

i = 1, 2, 4, 8, ..., n

⇒  $\log n$

#

int i = n

while (i ≥ 1) {

==

i = i / 2;

}

i = n, n/2, n/4, n/8, ..., 1

⇒  $\log n$

What will be the output of the following function?

```
void example() {  
    int x = 10;  
    for (int i = 1; i <= 5; i++) {  
        x -= 2;  
        if (x % 3 == 0) {  
            continue;  
        }  
        print(x);  
    }  
}
```

*Handwritten output:*

| i | x |
|---|---|
| 1 | 8 |
| 2 | 6 |
| 3 | 4 |
| 4 | 2 |
| 5 | 0 |

*Handwritten output (highlighted):*

8  
4  
2

example();

What will be the output of the following function?

```
void processLoop() {  
    for (int i = 1; i != 10; i += 2) {  
        print(i);  
    }  
}
```

*Handwritten output:*

i = 1, 3, 5, 7, 9, 11

processLoop();

What will be the output of the following function when executed?

```
void example() {  
    for (int i = 1; i <= 4; i++) {  
        if (i % 2 == 0 && i * 2 < 7) {  
            print(i);  
        }  
    }  
}  
  
example();
```

i  
1  
2  
3  
4

2

What is the output in the following function?

```
void find_sum() {  
    int sum;  
    for (int i = 1; i <= 5; i++) {  
        sum += i;  
    }  
    print(sum);  
}  
  
find_sum();
```

What will be the output of the following function?

```
void example2() {  
    for (int i = 1; i <= 3; i++) {  
        for (int j = 3; j >= 1; j--) {  
            print(i + " " + j + " ");  
        }  
        print("\n")  
    }  
}
```

example2();

| i | j       |
|---|---------|
| 1 | 3, 2, 1 |
| 2 | 3 2 1   |

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 1 | 3 | 1 | 2 | 1 | 1 |
| 2 | 3 | 2 | 2 | 2 | 1 |
| 3 | 3 | 3 | 2 | 3 | 1 |

What is the output in the following code?

```
void solve() {  
    int[] arr = {1, 2, 3, 4};  
    for (int i = 0; i <= arr.length; i++) {  
        print(arr[i]);  
    }  
}  
solve();
```

→ Index out of Bounds

## What issue exists in the following function?

```
void solve() {  
    int[] arr = {5, 10, 15, 20};  
    for (int i = arr.length; i >= 0; i--) {  
        print(arr[i]);  
    }  
}  
solve();
```